

The Need for Display Names

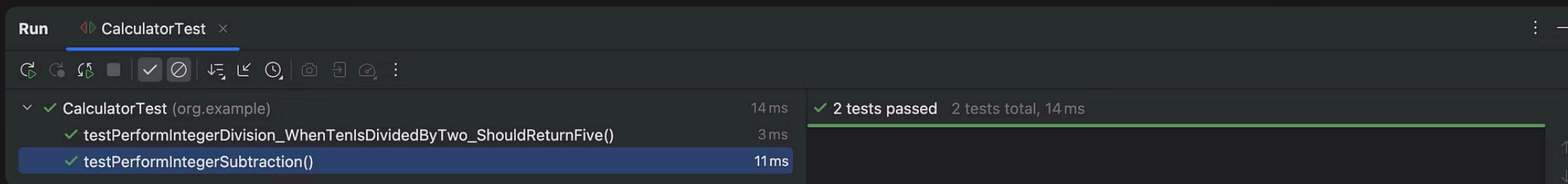


Test reports show **method names** exactly as **written** in the **code**. Long, technical names become **difficult** to **read** in **test reports**. **Display names** allow us to show **test descriptions** in a **cleaner**, more friendly **form**. This **improves readability** and helps **non-technical reviewers** understand **test intent**.

RP by **Rodrigo Martins Pagliares**

Last update: 11/26/2025

The test method names appears in the test report



We can tailor all these names shown in the output:

```
CalculatorTest
testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive
testPerformIntegerSubtraction
```

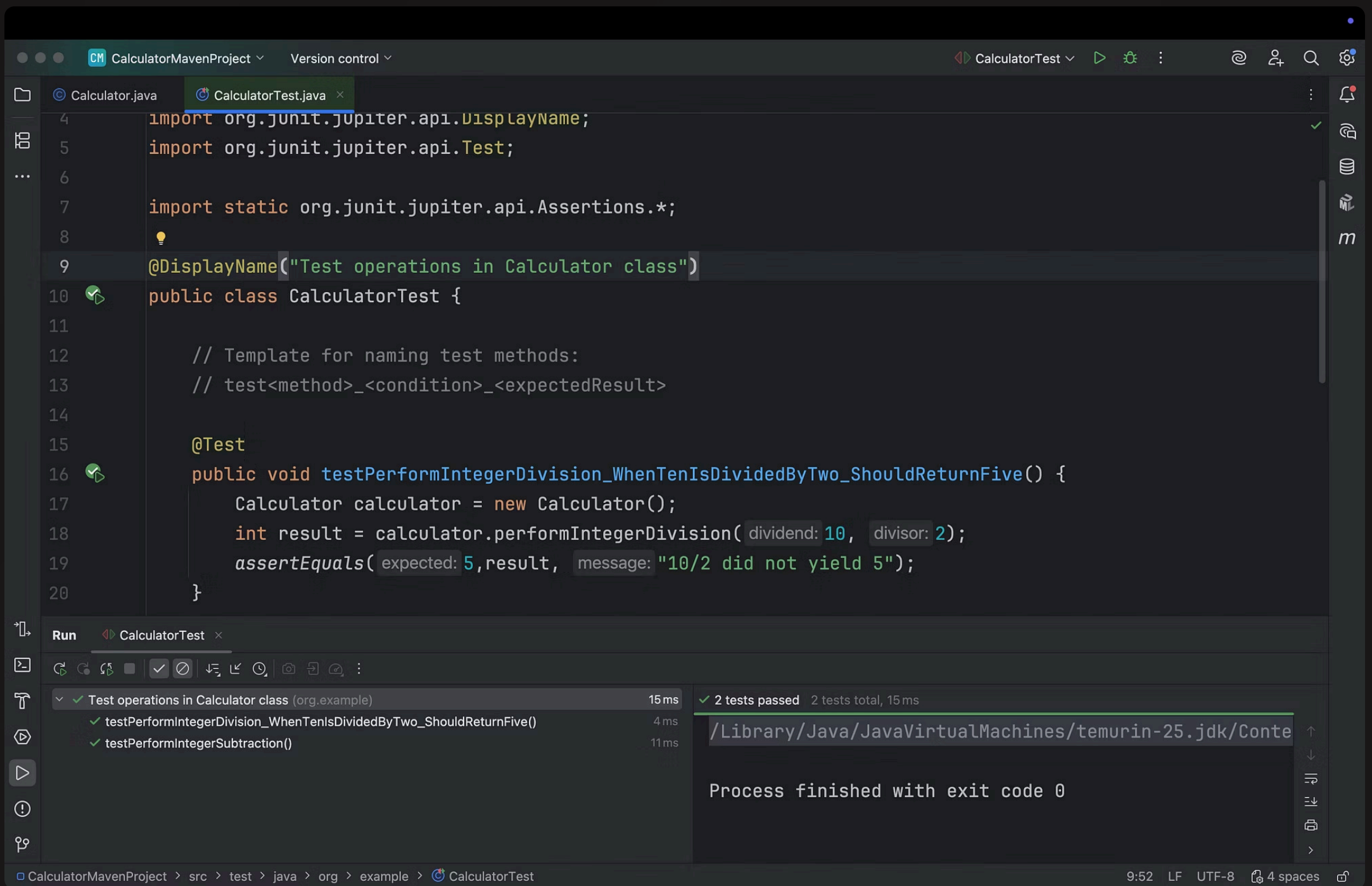
Using @DisplayName



- Apply `@DisplayName` above a **test class** to rename it in reports
- Apply it above **individual test methods** to provide descriptive labels
- **Display names** tolerate **spaces, punctuation, and natural language phrasing**
- **Replaces** long **technical names** with short, **meaningful descriptions**
- Example: `@DisplayName("Test 10 divided by 2")`

Using @DisplayName to produce better method names in test reports

Tailoring the name of the test class.



```
4 import org.junit.jupiter.api.DisplayName;
5 import org.junit.jupiter.api.Test;
6
7 import static org.junit.jupiter.api.Assertions.*;
8
9 @DisplayName("Test operations in Calculator class")
10 public class CalculatorTest {
11
12     // Template for naming test methods:
13     // test<method>_<condition>_<expectedResult>
14
15     @Test
16     public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
17         Calculator calculator = new Calculator();
18         int result = calculator.performIntegerDivision( dividend: 10, divisor: 2);
19         assertEquals( expected: 5, result, message: "10/2 did not yield 5");
20     }
```

Run CalculatorTest

Test Name	Duration
Test operations in Calculator class (org.example)	15 ms
testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive()	4 ms
testPerformIntegerSubtraction()	11 ms

✓ 2 tests passed 2 tests total, 15 ms

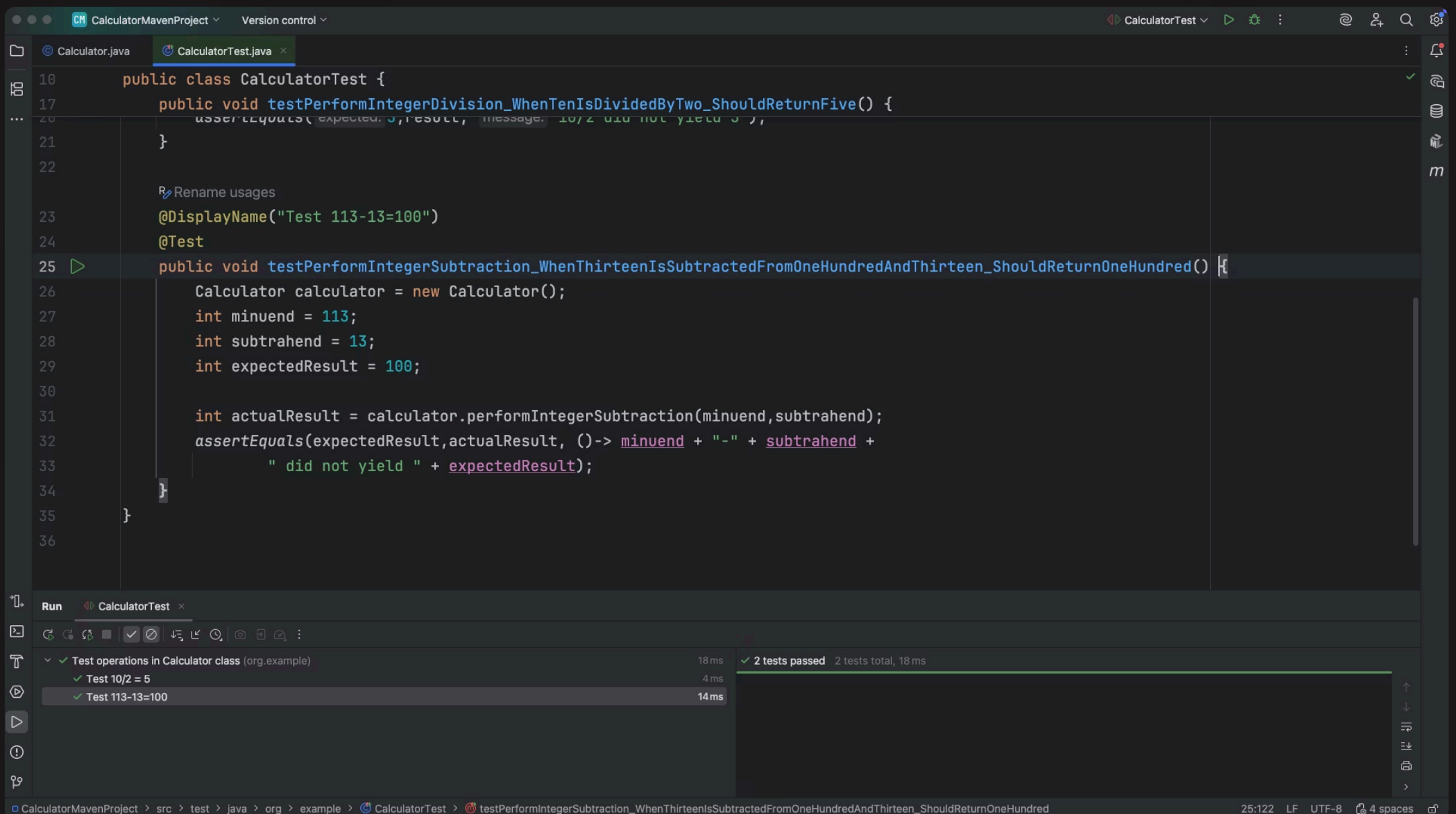
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Conte

Process finished with exit code 0

Notice our custom message in the output (**Test operations in Calculator class**)

Using @DisplayName to produce better method names in test reports

Tailoring the name of the test methods.



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The editor displays the file 'CalculatorTest.java' with the following code:

```
10 public class CalculatorTest {
17     public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
20         assertEquals(expected: 5, result, message: "10/2 did not yield 5");
21     }
22
23     @DisplayName("Test 113-13=100")
24     @Test
25     public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() {
26         Calculator calculator = new Calculator();
27         int minuend = 113;
28         int subtrahend = 13;
29         int expectedResult = 100;
30
31         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
32         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
33             " did not yield " + expectedResult);
34     }
35 }
36
```

Below the editor, the 'Run' tab shows the test results for 'CalculatorTest'. The report indicates that 2 tests passed out of 2 tests total, taking 18 ms. The specific tests and their durations are:

Test Name	Duration
Test 10/2 = 5	4 ms
Test 113-13=100	14 ms

The bottom status bar shows the file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred'.

Results in Test Reports



After adding `@DisplayName`, test reports become clearer and more readable

Both test class and test methods appear with friendly names

Helps communicate test purpose more effectively to other developers

Especially useful for reporting, onboarding, and code reviews

Conclusion

- `@DisplayName` is a simple but powerful tool for improving test readability
- Makes **test suites** more approachable, especially in large projects
- Allows **technical test names** to remain structured while **reports stay user-friendly**
- **Enhances communication** for both technical and non-technical stakeholders



Open Floor Discussion



Do you prefer descriptive test names in code, display names, or both?

In your current projects, would friendly test labels improve workflow?

How could display names support teaching or code review sessions?